BTS CIEL · PROGRAMMATION C++ · TRAVAUX PRATIQUES

TP04 — Module de conversion pour capteur de température LM35

Prérequis : chapitres 1 à 5

Contexte professionnel

Un client équipe ses armoires techniques d'une carte **Arduino Uno** reliée à un capteur de température **LM35**. Le LM35 délivre une tension proportionnelle à la température : **10 mV par degré Celsius**. Cette tension est lue sur l'entrée analogique A0 par le convertisseur analogique-numérique (CAN) de la carte, sur **10 bits** : il renvoie un entier de **0 à 1023**, pour une tension de référence de **5 V**.

Pour fiabiliser le logiciel embarqué, l'équipe décide de développer et de **tester sur PC** le module de conversion avant de l'intégrer au microcontrôleur. Le module ne doit donc contenir **aucun affichage ni aucune saisie** : uniquement des traitements, réutilisables tels quels sur la carte.

Objectifs

- Concevoir un module (`.h` + `.cpp`) à partir d'une spécification.
- Choisir le mode de passage de chaque paramètre.
- Écrire un programme de tests unitaires, valeurs limites comprises.
- Compiler un programme en plusieurs fichiers et interpréter les erreurs d'édition de liens.

Compétences mobilisées

Compétence	Mise en œuvre dans ce TP
C04 — Analyser un système informatique	Exploiter les caractéristiques du CAN et du capteur pour établir les formules
C05 — Concevoir un système informatique	Découper le logiciel en module réutilisable et programme principal
C06 — Valider un système informatique	Concevoir et exécuter les tests unitaires du module
C08 — Coder	Écrire le module, les tests et le programme principal

Spécification du module `capteur`

Formules

- Tension (V) = valeur CAN × 5,0 / 1023 ; ainsi, 0 donne 0 V et 1023 donne exactement 5 V.
- Température (°C) = tension (V) / 0,010.

Éléments à déclarer dans `capteur.h`

Élément	Rôle
`CAN_MAX`	Valeur maximale du CAN : 1023
`TENSION_REFERENCE_V`	Tension de référence : 5,0 V
`LM35_VOLTS_PAR_DEGRE`	Sensibilité du capteur : 0,010 V/°C
`struct Statistiques`	Champs `minimum`, `maximum`, `moyenne` (`double`)
`valeurCanValide`	Reçoit une valeur CAN (`int`) ; vraie si elle est comprise entre 0 et 1023 inclus
`canVersTension`	Reçoit une valeur CAN (`int`) ; renvoie la tension en volts
`tensionVersCelsius`	Reçoit une tension en volts ; renvoie la température en °C
`canVersCelsius`	Reçoit une valeur CAN ; renvoie la température, en réutilisant les deux fonctions précédentes
`calculerStatistiques`	Reçoit un `std::vector<double>` de températures et une `Statistiques` à remplir ; renvoie `false` si le vecteur est vide

Travail demandé

Partie 1 — Analyse

1. Calculez à la main la tension et la température pour les valeurs CAN 0, 45, 205 et 1023.
2. Quelle est la résolution du système, c'est-à-dire l'écart de température entre deux valeurs CAN consécutives ?
3. Pour chaque fonction de la spécification, écrivez le prototype complet et justifiez le mode de passage de chaque paramètre (valeur, référence, référence constante).

Partie 2 — Module

4. Écrivez `capteur.h` (garde d'inclusion, constantes, `struct`, prototypes) et `capteur.cpp` (définitions).

Partie 3 — Tests unitaires

5. Complétez le fichier de départ `test_capteur.cpp`, qui fournit une fonction d'affichage des verdicts : ajoutez au moins les tests du tableau ci-dessous.
6. Compilez avec `g++ -std=c++17 -Wall -Wextra test_capteur.cpp capteur.cpp -o test_capteur` et corrigez jusqu'à `0 test(s) en echec`.

Partie 4 — Programme principal

7. Écrivez `station.cpp`, qui :
   - lit des valeurs CAN saisies au clavier jusqu'à la valeur `-1` (on suppose les saisies numériques) ;
   - refuse avec un message toute valeur CAN invalide autre que `-1` ;
   - affiche la conversion de chaque valeur valide ;
   - affiche enfin les statistiques des températures, ou `Aucune mesure`.
8. Compilez `station.cpp` avec le module.

Partie 5 — Compilation et édition de liens

9. Compilez le module et le programme principal **séparément** (`-c`), puis assemblez les fichiers objets.
10. Retirez `capteur.cpp` de la commande de compilation : relevez le message, l'étape concernée et expliquez.
11. Déplacez la **définition** de `canVersTension` dans `capteur.h`, puis compilez `station.cpp capteur.cpp` : relevez le message et expliquez. Remettez le code en ordre.

Contraintes techniques

- Compilation avec `g++ -std=c++17 -Wall -Wextra`, sans erreur ni avertissement.
- `capteur.h` et `capteur.cpp` n'incluent pas `<iostream>` et n'affichent rien.
- Aucune valeur numérique « en dur » dans les formules : uniquement les constantes du module.
- Les résultats de type `double` sont testés avec une tolérance de `1e-6`.

Tests à effectuer

Tests unitaires (`test_capteur.cpp`)

Fonction	Entrée	Résultat attendu
`valeurCanValide`	`-1` / `0` / `1023` / `1024`	`false` / `true` / `true` / `false`
`canVersTension`	`0` / `1023`	`0.0` / `5.0`
`canVersTension`	`205`	`1.0019550` (à 1e-6 près)
`tensionVersCelsius`	`0.25`	`25.0`
`canVersCelsius`	`45`	`21.9941349` (à 1e-6 près)
`calculerStatistiques`	`{21.5, 23.0, 19.5}`	`true` ; min `19.5`, max `23.0`, moyenne `21.3333333` (à 1e-6 près)
`calculerStatistiques`	vecteur vide	`false`

Programme principal (`station.cpp`)

Saisies : `45`, `1500`, `51`, `41`, `-1`.

```
CAN 45 -> 0.219941 V -> 21.9941 C
Valeur CAN invalide : 1500
CAN 51 -> 0.249267 V -> 24.9267 C
CAN 41 -> 0.200391 V -> 20.0391 C
Minimum : 20.0391 C
Maximum : 24.9267 C
Moyenne : 22.32 C
```

Livrables

- `capteur.h`, `capteur.cpp`, `test_capteur.cpp`, `station.cpp`.
- La feuille d'analyse (partie 1), les messages relevés et les explications des questions 10 et 11.

Critères d'évaluation

Critère	Indicateurs de réussite	Points
Analyse	Calculs exacts, résolution correcte, passages de paramètres justifiés	4
Module	En-tête complet (garde, constantes, `struct`, prototypes), définitions exactes, aucune entrée-sortie	5
Tests unitaires	Tous les cas du tableau, valeurs limites, tolérance sur les `double`, `0 test(s) en echec`	5
Programme principal	Saisie jusqu'à -1, valeurs invalides refusées, statistiques ou `Aucune mesure`	3
Compilation séparée	Messages relevés et expliqués (étape, cause)	3
Total		**20**